

Introduction to Pandas

Resources:

- [Official Pandas User Guide](#)
- [Python for Data Analysis by Wes McKinney](#)

Why use Pandas?

While NumPy is excellent for numerical matrices (homogeneous data), Pandas is designed for tabular data (heterogeneous data). Think of a DataFrame as a NumPy array with labels, or an Excel sheet, allowing you to mix strings, integers, and floats in one table.

```
In [... import numpy as np
import pandas as pd
```

```
In [... # why pandas? it's useful!
foo = pd.read_html('https://arghyadutta.github.io/not
foo.to_excel('try.xlsx')
```

Pandas Series

A 1D labeled array that can hold any data type (ints, floats, strings, objects).

- Like a NumPy 1D array, but with an index
- Each value is accessed by its label, not just position
- Supports vectorized operations and alignment by index

```
In [... foo = [10, 20, 30]
foo[0]
```

```
Out[... 10
```

```
In [... s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])

# s = pd.Series({'a':10,'b':20,'c':30})
print(s)
# the values of a series are returned as numpy arrays
```

```
print(s, s.values, type(s.values))

# vectorized operation (often intuitive)
s + 5
```

```
a    10
b    20
c    30
dtype: int64
a    10
b    20
c    30
dtype: int64 [10 20 30] <class 'numpy.ndarray'>
```

```
Out[... a    15
        b    25
        c    35
        dtype: int64
```

DataFrame

2D, labeled tabular data structure.

- Rows and columns both have indexes
- Columns can hold different data types
- Supports vectorized operations, alignment, grouping, merging

The diagram illustrates a DataFrame as a 2D table. The columns are labeled 'Name', 'Team', 'Number', 'Position', 'Age', 'Height', 'Weight', 'College', and 'Salary'. The rows are indexed from 0 to 6. A 'Missing value' (NaN) is shown in the 'Number' column for row 3 and the 'Age' column for row 5. The 'Data' label points to the body of the table.

	Name	Team	Number	Position	Age	Height	Weight	College	Salary
0	Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0
1	John Holland	Boston Celtics	30.0	SG	27.0	6-5	205.0	Boston University	NaN
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0	6-10	231.0	NaN	5000000.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0	6-8	235.0	LSU	1170960.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0	6-2	190.0	Louisville	1824360.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN	6-9	260.0	Ohio State	2569260.0
6	Evan Turner	Boston Celtics	11.0	SG	27.0	6-7	220.0	Ohio State	3425510.0

Source: Wikimedia commons

```
In [... # defining a dataframe from a dictionary
data = {
    'name': ['Alice', 'Bob', 'Charlie'],
    'age': [25, 30, 35],
    'score': [88.5, 92.0, 79.5]}
```

```

}

df = pd.DataFrame(data)
print("---- Original DataFrame ----")
print(df)

# print("the structure of your data")
# df.info()

print("get some statistical values")
print(df.describe())

print(f"\nColumns: {df.columns.tolist()}")

```

```

---- Original DataFrame ----
   name  age  score
0  Alice   25   88.5
1    Bob   30   92.0
2 Charlie   35   79.5
get some statistical values

```

	age	score
count	3.0	3.000000
mean	30.0	86.666667
std	5.0	6.448514
min	25.0	79.500000
25%	27.5	84.000000
50%	30.0	88.500000
75%	32.5	90.250000
max	35.0	92.000000

```
Columns: ['name', 'age', 'score']
```

Indexing: loc vs iloc

- `.loc` : Label-based (uses names/labels)
- `.iloc` : Integer-position based (uses 0, 1, 2...)

```

In [...] # row
df
df.loc[0]['name'] # label-based
df.loc[0,'name'] # label-based
df.iloc[0,0] # position-based

```

```
# column
df.iloc[:,1]
df["age"]
df[["name", "score"]]
```

```
Out[...      name  age  score
0    Alice   25   88.5
1     Bob   30   92.0
2  Charlie   35   79.5
```

```
Out[...  'Alice'
```

```
Out[...  'Alice'
```

```
Out[...  'Alice'
```

```
Out[...  0    25
1    30
2    35
Name: age, dtype: int64
```

```
Out[...  0    25
1    30
2    35
Name: age, dtype: int64
```

```
Out[...      name  score
0    Alice   88.5
1     Bob   92.0
2  Charlie   79.5
```

```
In [...  # boolean indexing works as in numpy
df
df[df['age'] > 28]
df[df['score'] > 90]
df.query("score >= 90")

# more complex logical operation

print(df[(df['age'] > 25) & (df['score'] > 80)])
```

```
Out [...]
```

	name	age	score
0	Alice	25	88.5
1	Bob	30	92.0
2	Charlie	35	79.5

```
Out [...]
```

	name	age	score
1	Bob	30	92.0
2	Charlie	35	79.5

```
Out [...]
```

	name	age	score
1	Bob	30	92.0

```
Out [...]
```

	name	age	score
1	Bob	30	92.0

	name	age	score
1	Bob	30	92.0

Operations and column manipulations

```
In [...]
```

```
# New column
new_df = df.copy()
# id(new_df)
# id(df)
new_df['random_col'] = [1,2,3]
new_df["passed"] = new_df["score"] >= 85
new_df["old"] = new_df["age"] > 25 #why 25? nobody kn
new_df

# Vectorized (preferred)
new_df["age_squared"] = new_df["age"] ** 2

new_df.sort_values("age", ascending=False)
```

```
Out [...]
```

	name	age	score	random_col	passed	old
0	Alice	25	88.5	1	True	False
1	Bob	30	92.0	2	True	True
2	Charlie	35	79.5	3	False	True

```
Out[...      name age score random_col passed old age_squared
```

	name	age	score	random_col	passed	old	age_squared
2	Charlie	35	79.5	3	False	True	1225
1	Bob	30	92.0	2	True	True	900
0	Alice	25	88.5	1	True	False	625

```
In [... new_df.head(3) #first two rows
new_df.tail(2) # last two rows

# how many unique values in a particular column?
new_df.value_counts("name")
```

```
Out[...      name age score random_col passed old age_squared
```

	name	age	score	random_col	passed	old	age_squared
0	Alice	25	88.5	1	True	False	625
1	Bob	30	92.0	2	True	True	900
2	Charlie	35	79.5	3	False	True	1225

```
Out[...      name age score random_col passed old age_squared
```

	name	age	score	random_col	passed	old	age_squared
1	Bob	30	92.0	2	True	True	900
2	Charlie	35	79.5	3	False	True	1225

```
Out[... name
Alice      1
Bob        1
Charlie    1
Name: count, dtype: int64
```

Missing Data

```
In [... # # uncomment, and run. explore more!
# df_nan = df.copy()
# df_nan.loc[1, 'score'] = np.nan
# df_nan
# df_nan.isna()
# df_nan.fillna(df_nan['score'].mean())
```

GroupBy

```
In [... # df['department'] = ['CS', 'Math', 'CS']
```

```
# df
# df.groupby('department')['score'].mean()
```

Merge

```
In [...] info = pd.DataFrame({
    'name': ['Charlie', 'Alice', 'Bob'],
    'city': ['Chennai', 'Delhi', 'Mumbai']
})
df
info
pd.merge(df, info, on='name')
```

```
Out [...]
   name  age  score
0  Alice   25   88.5
1   Bob   30   92.0
2 Charlie   35   79.5
```

```
Out [...]
   name  city
0 Charlie Chennai
1  Alice   Delhi
2   Bob   Mumbai
```

```
Out [...]
   name  age  score  city
0  Alice   25   88.5  Delhi
1   Bob   30   92.0  Mumbai
2 Charlie   35   79.5  Chennai
```

Data read and write

```
In [...] df.to_csv("students.csv", index=False)
df.to_excel("students.xlsx", index=False)
```

```
In [...] new_df2 = pd.read_excel('students.xlsx')
new_df2
# pd.to_csv
```

Out [...]

	name	age	score
0	Alice	25	88.5
1	Bob	30	92.0
2	Charlie	35	79.5

In [...]

```
# source: https://www.programiz.com/python-programmin

# create a dictionary containing the data
data = {'Category': ['Electronics', 'Clothing', 'Elec
          'Sales': [1000, 500, 800, 300]}

# create a DataFrame using the data dictionary
df = pd.DataFrame(data)
df
# group the DataFrame by the Category column and
# calculate the sum of Sales for each category
grouped = df.groupby('Category')['Sales'].mean()

# print the grouped data
print(grouped)
```

Out [...]

	Category	Sales
0	Electronics	1000
1	Clothing	500
2	Electronics	800
3	Clothing	300

```
Category
Clothing      400.0
Electronics   900.0
Name: Sales, dtype: float64
```

In [...]